6-1 · 실시간 데이터와 Kafka · 정리·회고 0.5H

정리·회고

용어 여덟 개를 우체국으로, 6-2 준비 확인

이 시간의 목표

30분 뒤에는

① 용어 여덟 개를 우체국 비유로 한 장에 정리한다

② CLI 명령 일곱 개를 「하려는 일」로 떠올린다

③ 자주 겪는 오류 다섯 가지의 원인과 해결을 안다

④ docker rm -fv kafka 뒤 이미지 · 메모리로 6-2 준비를 확인한다

get-offsets 결과를 채팅에 올려 비교해 보세요 — 키 없는 메시지가 모인 칸은 사람마다 다릅니다

오늘의 요약

용어 여덟 개 — 우체국 비유와 한 줄 정의

용어	우체국 비유	한 줄 정의
브로커	우체국 건물	메시지를 저장하고 전달하는 서버
토픽	사서함 벽	이벤트 종류별 통로
파티션	개별 사서함	토픽을 나눈 칸 · 병렬과 순서의 단위
오프셋	편지 일련번호	파티션 안의 위치
Producer	편지 부치는 사람	메시지를 보내는 쪽
Consumer	편지 찾으러 오는 사람	메시지를 읽는 쪽
컨슈머 그룹	부서	같은 그룹 나눠 읽기 · 다른 그룹 각자
KRaft	—	ZooKeeper 없이 도는 최신 구조

Kafka 의 편지는 꺼내 읽어도 사라지지 않는다 — 여러 부서가 각자 자기 속도로 같은 편지를 읽는다

오늘의 요약

CLI 명령 일곱 개 — 하려는 일로 찾기

하려는 일	명령
토픽 만들기	kafka-topics.sh --create --topic T --partitions N
토픽 목록	kafka-topics.sh --list
토픽 상세	kafka-topics.sh --describe --topic T
보내기	kafka-console-producer.sh --topic T
읽기	kafka-console-consumer.sh --topic T [--from-beginning]
키 함께	--property parse.key=true --property key.separator=:
오프셋 확인	kafka-get-offsets.sh --topic T

앞에 docker exec kafka /opt/kafka/bin/ 과 --bootstrap-server localhost:9092 를 붙인다 · 단일 브로커는 --replication-factor 1

오늘의 체크리스트

여덟 개를 스스로 확인해 보세요

1. 배치와 스트리밍의 차이를 처리 단위·지연 관점에서 설명했다
2. 「늦게 알면 가치가 없어지는가」로 스트리밍 필요 여부를 판단한다
3. 이벤트 시간과 처리 시간의 차이를 설명할 수 있다
4. 토픽·파티션·오프셋을 우체국 비유로, 순서는 파티션 안에서만
5. Docker 로 단일 브로커를 기동하고 토픽을 만들어 송수신했다
6. Consumer 를 재시작해도 이전 메시지가 남아 있음을 확인했다
7. --from-beginning 유무로 읽는 시작점이 달라지는 것을 봤다
8. 같은 키 → 같은 파티션, kafka-get-offsets.sh 로 오프셋이 느는 것을 봤다

자주 겪는 오류

증상으로 찾는 원인과 해결

증상	원인	해결
docker run 이 Conflict · port is already allocated	같은 이름 컨테이너 · 9092 를 다른 것이 씀	docker rm -fv kafka · 실습점검 2단계로 찾기
WARN Connection to node -1 could not be established	브로커가 아직 뜨는 중 이거나 죽어 있음	뜨는 중이면 저절로 이어감 · 계속되면 docker logs
호스트 CLI·GUI 접속은 되는데 송수신 끊김	advertised.listeners 가 알려 준 주소로 재접속	오늘은 docker exec 로 · 리스너는 6-2 에서
replication-factor 2 토픽이 안 만들어짐	브로커 1대 — 복제본 둘 곳 없음	단일 브로커는 1
No key separator found on line number 1	parse.key 인데 콜론 없는 줄	Producer 다시 켜고 키:값 으로

오늘은 docker exec 로 컨테이너 안에서 실습하면 리스너 문제가 없다 — 리스너 설정은 6-2 에서 정식으로

예상 질문

자주 나오는 질문 네 가지

1. **Kafka 는 데이터베이스인가요?**
   아니다 — 조회·수정이 아니라 「흘려보내고 보관하는」 것이 목적. 보존 기간 동안 되감아 읽을 수는 있다

2. **메시지는 얼마나 보관되나요?**
   기본 7일 — 토픽마다 설정할 수 있고, 용량 기준으로도 정한다

3. **RabbitMQ 같은 큐와 뭐가 다른가요?**
   큐는 읽으면 사라지고, Kafka 는 남는다 — 그래서 여러 소비자가 각자 읽을 수 있다

4. **파티션은 몇 개가 적당한가요?**
   동시에 읽을 컨슈머 수만큼이 출발점 — 늘릴 수는 있지만 줄일 수는 없다

다음 차시(6-2)를 위한 준비

오늘 컨테이너는 지우고, 이미지와 메모리는 확인

1. **docker rm -fv kafka**
   6-2 에서 compose 로 다시 만든다 — -v 가 이름 없는 볼륨 3개까지 지운다

2. **docker images | grep kafka**
   apache/kafka:3.7.0 이미지는 남겨 둔다(612MB)

3. **docker info | grep -i "total memory"**
   7.7GiB 안팎 이상(설정 8GB, 실측 7.749GiB) — 6-2 부터 컨테이너가 늘어난다

4·5주차 컨테이너가 떠 있으면

docker stop 으로 내려 두기

다음 회차 예고

6-2 · Docker 기반 Kafka 클러스터

01

이론 1H

KRaft 필수 설정과

리스너 2계층 구조

02

실습 2.5H

토픽 관리 + 메시지 흐름·

컨슈머 그룹 검증

03

정리 0.5H

compose 파일 리뷰

체크리스트·질의응답

준비: docker rm -fv kafka 로 9092 비우기 · 작업 폴더 week6/ · 이번 주 내내 쓸 docker-compose.yml 하나로 관리

6-1 · 정리·회고 0.5H 마무리

다음 회차는 6-2 Docker 기반 Kafka 클러스터

Docker Compose 로 KRaft 단일 브로커를 정식 구성하고, 리스너 2계층과 컨슈머 그룹·랙을 다룹니다

- ✓ **docker-compose.yml 로 KRaft 단일 브로커**
- ✓ **listeners 와 advertised.listeners 의 차이**
- ✓ **토픽 생성·조회·설정 변경 CLI**
- ✓ **kafka-consumer-groups.sh 로 랙 확인**

준비할 것

docker rm -fv kafka · week6/ 폴더